

Búsqueda de Ítems Similares

Locality-Sensitive Hashing

Minería de Datos

Universidad del Norte

- 1 Introducción y motivación
- 2 Similitud de Jaccard
- 3 Shingling
- 4 MinHashing
- 5 Locality-Sensitive Hashing
- 6 Más allá de Jaccard

Necesitamos encontrar ítems **similares** en colecciones masivas:

- **Detección de plagio** — documentos con contenido sustancialmente idéntico
- **Páginas espejo en la web** — copias del mismo contenido en distintos servidores
- **Artículos de noticias similares** — misma fuente (e.g., AP) en distintos periódicos
- **Búsqueda visual** — imágenes similares en una base de datos masiva
- **Bases de datos vectoriales / RAG** — fragmentos de texto semánticamente relevantes

El problema fundamental

Enfoque ingenuo

Comparar **todos los pares** de n documentos: $\binom{n}{2} = O(n^2)$ comparaciones.

Para $n = 1\,000\,000$ documentos:

$$\binom{10^6}{2} \approx 5 \times 10^{11} \text{ comparaciones — inviable}$$

Objetivo

Enfocar la atención solo en pares *probablemente* similares, sin examinar todos los pares posibles.

Pipeline de tres pasos



- ❶ **Shingling:** convertir documentos en representaciones de conjuntos
- ❷ **MinHashing:** comprimir conjuntos grandes en firmas cortas que preservan similitud
- ❸ **LSH:** identificar pares candidatos con alta probabilidad de ser similares

Similitud de Jaccard

Definición

Dados dos conjuntos A y B :

$$\text{SIM}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Propiedades:

- $\text{SIM}(A, B) \in [0, 1]$
- $\text{SIM}(A, B) = 0 \iff A \cap B = \emptyset$
- $\text{SIM}(A, B) = 1 \iff A = B$

Ejemplo

$$A = \{1, 2, 3, 4\}, B = \{2, 3, 5, 7\}$$

$$A \cap B = \{2, 3\}, \quad A \cup B = \{1, 2, 3, 4, 5, 7\}$$

$$\text{SIM}(A, B) = 2/6 = 1/3 \approx 0,33$$

Definición

$$d_J(A, B) = 1 - \text{SIM}(A, B)$$

Es una **métrica válida**:

- $d_J(A, B) \geq 0$, con igualdad sii $A = B$
- $d_J(A, B) = d_J(B, A)$
- $d_J(A, C) \leq d_J(A, B) + d_J(B, C)$ (desigualdad triangular)

Definición

Un k -**shingle** (o k -grama) de un documento es cualquier subcadena de k caracteres consecutivos. El conjunto de k -shingles de D se denota $S_k(D)$.

Ejemplo: $D = \text{"abcdabd"} , k = 2$

$$S_2(D) = \{ab, bc, cd, da, bd\}$$

("ab" aparece dos veces, pero como *conjunto* solo se incluye una vez)

Intuición: documentos con texto similar comparten la mayoría de sus k -shingles $\Rightarrow$ alta similitud de Jaccard.

Elección de k y hashing

Elección de k :

Si el alfabeto tiene c caracteres, hay c^k shingles posibles.

- $k = 5$: $27^5 \approx 1,4 \times 10^7$ — adecuado para emails
- $k = 9$ o 10 : adecuado para documentos largos (artículos, web)

Regla: c^k debe ser $\gg$ longitud típica del documento.

Hashing de shingles:

En lugar de almacenar cadenas, aplicamos una función hash:

shingle (cadena) $\xrightarrow{\text{hash}}$ entero de 32 bits

Representamos cada documento como un **conjunto de enteros**.

Reduce memoria y facilita comparaciones.

El problema de escala

Los conjuntos de shingles pueden ser **enormes**:

- Un documento de n caracteres tiene hasta $n - k + 1$ shingles
- El universo de shingles posibles puede tener millones de elementos

Necesidad

Comprimir conjuntos grandes en **firmas cortas** (vectores de ~ 100 – 200 enteros) que *preservan la similitud de Jaccard*.

Esto es exactamente lo que hace **MinHashing**.

Matriz característica

Representamos una colección de conjuntos como una **matriz booleana**:

- **Filas:** elementos del universo (shingles)
- **Columnas:** conjuntos (documentos)
- Entrada $(i, j) = 1$ si el elemento $i \in$ conjunto j

Ejemplo

$$S_1 = \{a, d\}, S_2 = \{c\}$$

$$S_3 = \{b, d, e\}, S_4 = \{a, c, d\}$$

	S_1	S_2	S_3	S_4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

$\text{SIM}(S_1, S_4) = |\{a, d\}| / |\{a, c, d\}| = 2/3$ — matriz típicamente muy **dispersa**.

Definición de MinHash

MinHash

Dada una matriz característica, sea π una **permutación aleatoria** de sus filas. Para un conjunto S :

$$h_{\pi}(S) = \min_{i \in S} \pi(i)$$

Es la **primera fila** (en el orden de π) donde la columna de S tiene un 1.

Intuición: permutamos las filas aleatoriamente y buscamos, de arriba hacia abajo, la primera fila donde S “aparece” (tiene un 1).

El valor minhash es un **resumen** del conjunto S , determinado por la permutación π .

Ejemplo: cálculo de la firma MinHash

Elem	Permutación			Matriz característica				
	π_1	π_2	π_3	idx	S_1	S_2	S_3	S_4
<i>a</i>	3	4	1	1	1	0	0	1
<i>b</i>	1	3	4	2	0	0	1	0
<i>c</i>	5	1	2	3	0	1	0	1
<i>d</i>	4	2	5	4	1	0	1	1
<i>e</i>	2	5	3	5	0	0	1	0

$h_\pi(S)$ = índice mínimo de π entre los elementos de S

Matriz de firmas M

$$h_\pi(S) = \min_{x \in S} \pi(x)$$

	S_1	S_2	S_3	S_4
π_1	3	5	1	3
π_2	2	1	2	1
π_3	1	2	3	1

Ejemplo: $h_{\pi_1}(S_3)$

$S_3 = \{b, d, e\} \rightarrow \text{idx } \{2, 4, 5\}$
 $\min(\pi_1(2), \pi_1(4), \pi_1(5))$
 $= \min(1, 4, 2) = 1$

Más ejemplos: cálculo del minhash

Referencia

Elem	π_1	π_2	π_3	idx
<i>a</i>	3	4	1	1
<i>b</i>	1	3	4	2
<i>c</i>	5	1	2	3
<i>d</i>	4	2	5	4
<i>e</i>	2	5	3	5

$$h_{\pi}(S) = \min_{x \in S} \pi(x)$$

$$h_{\pi_2}(S_1): S_1 = \{a, d\} \rightarrow \text{idx } \{1, 4\}$$

$$\min(\pi_2(1), \pi_2(4)) = \min(4, 2) = 2$$

$$h_{\pi_2}(S_4): S_4 = \{a, c, d\} \rightarrow \text{idx } \{1, 3, 4\}$$

$$\min(\pi_2(1), \pi_2(3), \pi_2(4)) = \min(4, 1, 2) = 1$$

$$h_{\pi_3}(S_3): S_3 = \{b, d, e\} \rightarrow \text{idx } \{2, 4, 5\}$$

$$\min(\pi_3(2), \pi_3(4), \pi_3(5)) = \min(4, 5, 3) = 3$$

$$h_{\pi_3}(S_4): S_4 = \{a, c, d\} \rightarrow \text{idx } \{1, 3, 4\}$$

$$\min(\pi_3(1), \pi_3(3), \pi_3(4)) = \min(1, 2, 5) = 1$$

¿Para qué sirven los minhashes?

Ya sabemos calcular $h_{\pi}(S)$. La pregunta clave es:

¿Qué nos dice que dos conjuntos tengan el mismo minhash?

Intuición: si A y B son muy similares (comparten muchos elementos), entonces bajo una permutación aleatoria es **probable** que su elemento mínimo sea el mismo.

$S_1 = \{a, d\}$ y $S_4 = \{a, c, d\}$

$\text{SIM}(S_1, S_4) = 2/3$ (similares)

$h_{\pi_1}(S_1) = 3, \quad h_{\pi_1}(S_4) = 3 \checkmark$

$h_{\pi_3}(S_1) = 1, \quad h_{\pi_3}(S_4) = 1 \checkmark$

Coinciden en 2 de 3 permutaciones

$S_1 = \{a, d\}$ y $S_2 = \{c\}$

$\text{SIM}(S_1, S_2) = 0$ (distintos)

$h_{\pi_1}(S_1) = 3, \quad h_{\pi_1}(S_2) = 5 \times$

$h_{\pi_2}(S_1) = 2, \quad h_{\pi_2}(S_2) = 1 \times$

No coinciden en ninguna permutación

Idea: usar la **fracción de minhashes que coinciden** como estimador de la similitud de Jaccard. Esto requiere demostrar que no es casualidad...

¿Cuándo coinciden dos minhashes?

Dados A y B , permutamos las filas con π y recorremos de arriba hacia abajo.

$h_\pi(A)$ es la primera fila con un 1 en la columna A .

$h_\pi(B)$ es la primera fila con un 1 en la columna B .

Observación clave

$h_\pi(A) = h_\pi(B) \iff$ la **primera fila** (en el orden π) que tiene un 1 en *alguna* de las dos columnas, tiene un 1 en **ambas**.

Cada fila puede clasificarse según sus valores en las columnas A y B :

Col. A	Col. B	Tipo	Significado
1	1	X	elemento en $A \cap B$ — ambos tienen 1
1	0	Y	solo uno tiene 1: está en $A \triangle B$
0	1		
0	0	Z	ninguno tiene 1 — irrelevante

$$\Pr[h_\pi(A) = h_\pi(B)] = \text{SIM}(A, B)$$

Las filas tipo **Z** (ambas columnas en 0) nunca son “la primera con un 1”. Las ignoramos.

Solo importan las filas tipo X e Y. Al recorrer en orden π , la **primera fila relevante** que encontremos determina todo:

- Si es tipo **X** (ambos tienen 1): $h_\pi(A) = h_\pi(B)$ ✓
- Si es tipo **Y** (solo uno tiene 1): $h_\pi(A) \neq h_\pi(B)$ ✗

Como π es **uniforme**, cada fila relevante tiene la misma probabilidad de quedar primera:

$$\Pr[\text{primera fila relevante sea tipo X}] = \frac{\# \text{ filas tipo X}}{\# \text{ filas tipo X} + \# \text{ filas tipo Y}} = \frac{|A \cap B|}{|A \cup B|}$$

Teorema (Propiedad fundamental de MinHash)

$$\Pr[h_\pi(A) = h_\pi(B)] = \frac{|A \cap B|}{|A \cup B|} = \text{SIM}(A, B)$$

Propiedad fundamental — Ejemplo

Verifiquemos con $S_1 = \{a, d\}$ y $S_4 = \{a, c, d\}$: Filas X: $\{a, d\}$, Y: $\{c\}$, Z: $\{b, e\}$ (ignoradas)
Con la permutación $\pi = (b, e, a, d, c)$:

	S_1	S_4	Tipo
a	1	1	X
b	0	0	Z
c	0	1	Y
d	1	1	X
e	0	0	Z

Orden π	S_1	S_4	Tipo
b	0	0	Z
e	0	0	Z
a	1	1	X ← 1ra relevante
d	1	1	X
c	0	1	Y

Primera fila relevante: a (tipo X)

$$\Rightarrow h_\pi(S_1) = h_\pi(S_4) = a \checkmark$$

Verificación

$$\Pr[h_\pi(S_1) = h_\pi(S_4)] = \frac{|X|}{|X|+|Y|} = \frac{2}{2+1} = \frac{2}{3} = \text{SIM}(S_1, S_4) \checkmark$$

De un minhash a una firma

Acabamos de probar que para cualquier par (A, B) y permutación aleatoria π :

$$\Pr[h_\pi(A) = h_\pi(B)] = \text{SIM}(A, B)$$

Definamos la variable aleatoria $X_\pi = \mathbf{1}[h_\pi(A) = h_\pi(B)]$:

- $X_\pi = 1$ (“coinciden”) con probabilidad $p = \text{SIM}(A, B)$
- $X_\pi = 0$ (“no coinciden”) con probabilidad $1 - p$

Esto es un **experimento de Bernoulli** con parámetro desconocido $p = \text{SIM}(A, B)$.

Problema: un experimento no alcanza

$X_\pi \in \{0, 1\}$ solo nos dice “coinciden” o “no”. Es como lanzar una moneda sesgada **una sola vez**: no podemos inferir el sesgo p .

Solución: repetir con m permutaciones independientes $\pi_1, \dots, \pi_m$ y estimar p por la fracción de coincidencias:

$$\hat{p} = \frac{|\{i : h_{\pi_i}(A) = h_{\pi_i}(B)\}|}{m} \approx \text{SIM}(A, B)$$

Estos m valores $h_{\pi_i}(S)$ forman la **firma MinHash** de S .

Definición

Dadas m permutaciones independientes, la **firma** de un conjunto S es:

$$\text{sig}(S) = [h_1(S), h_2(S), \dots, h_m(S)]$$

Definimos variables indicadoras i.i.d.: $X_i = \mathbf{1}[h_i(A) = h_i(B)]$, con $\Pr[X_i = 1] = \text{SIM}(A, B)$.

Nuestro estimador es el **promedio muestral**:

$$\hat{p} = \frac{1}{m} \sum_{i=1}^m X_i = \frac{|\{i : h_i(A) = h_i(B)\}|}{m}$$

¿Por qué funciona? — Ley de los grandes números

Ley de los grandes números

Si $X_1, \dots, X_m$ son v.a. i.i.d. con $E[X_i] = p$, entonces:

$$\frac{1}{m} \sum_{i=1}^m X_i \xrightarrow{m \rightarrow \infty} p$$

En nuestro caso: cada $X_i = \mathbf{1}[h_i(A) = h_i(B)]$ es Bernoulli con $p = \text{SIM}(A, B)$.

⇒ La fracción de componentes iguales en las firmas **converge** a $\text{SIM}(A, B)$:

$$\hat{p} = \frac{|\{i : h_i(A) = h_i(B)\}|}{m} \xrightarrow{m \rightarrow \infty} \text{SIM}(A, B)$$

- A mayor $m \Rightarrow$ mejor estimación, pero más costo
- Valores típicos: $m = 100\text{--}200$
- **Compresión:** de conjuntos con miles de elementos a vectores de m enteros

Simulación: experimento de Bernoulli

Configuración del experimento

Dos conjuntos concretos

$$A = \{1, 2, \dots, 10\}$$

$$B = \{5, 6, \dots, 14\}$$

$$A \cap B = \{5, 6, 7, 8, 9, 10\}$$

$$A \cup B = \{1, 2, \dots, 14\}$$

$$\text{SIM}(A, B) = \frac{6}{14} \approx 0.4286$$

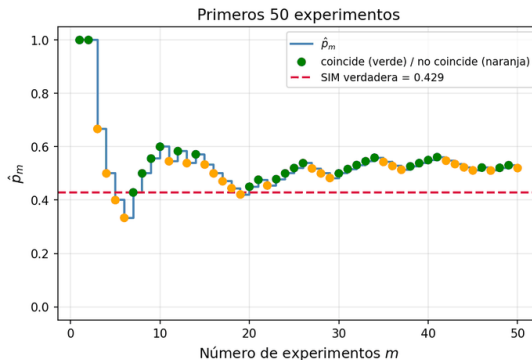
Un experimento MinHash

- 1 Generar permutación aleatoria π del universo $\{1, \dots, 20\}$
- 2 Calcular $h_\pi(A) = \min_{a \in A} \pi(a)$ y $h_\pi(B) = \min_{b \in B} \pi(b)$
- 3 Registrar $X = \mathbf{1}[h_\pi(A) = h_\pi(B)]$

Repetir m veces con permutaciones independientes y estimar:

$$\hat{p}_m = \frac{1}{m} \sum_{i=1}^m X_i$$

Simulación: convergencia al valor real



Izquierda: 500 experimentos — la estimación se estabiliza alrededor de $\text{SIM} = 0,429$.

Derecha: primeros 50 experimentos — cada punto verde es una coincidencia ($X = 1$), naranja es $X = 0$.

Implementación práctica — el problema

En teoría, cada función minhash h_i requiere una **permutación aleatoria** π_i de todas las filas.

Problema

Si el universo tiene N elementos (filas), almacenar una permutación requiere $O(N)$ espacio. Para m funciones minhash: $O(mN)$ espacio solo para las permutaciones.

Ejemplo: con k -shingles de caracteres, el universo tiene $N = |\Sigma|^k$ shingles posibles. Para $\Sigma = \text{ASCII}$ (128 caracteres) y $k = 4$: $N = 128^4 \approx 2,7 \times 10^8$.

Una permutación de N elementos = un arreglo de N enteros (cada uno indica a dónde va la fila i). A 4 bytes por entero:

$$m \times N \times 4 \text{ bytes} = 100 \times 2,7 \times 10^8 \times 4 \approx 100 \text{ GB solo en permutaciones}$$

Además, generar y aplicar permutaciones completas es lento: $O(N)$ por permutación.

¿Podemos simular una permutación sin almacenarla explícitamente?

Congruencia módulo p : definición

Definición

Dados enteros a, b y un entero positivo n , decimos que a es **congruente con b módulo n** si $n \mid (a - b)$, es decir, si $a - b$ es múltiplo de n . Se escribe:

$$a \equiv b \pmod{n}$$

Interpretación: $a \equiv b \pmod{n}$ significa que a y b dejan el **mismo residuo** al dividir por n .

¿Por qué “mismo residuo”?

Si $a = qn + r$ y $b = q'n + r$ (mismo residuo r), entonces $a - b = (q - q')n$, que es múltiplo de n . ✓

Recíprocamente, si $n \mid (a - b)$, entonces a y b tienen el mismo residuo módulo n .

Congruencia módulo p : ejemplos

Tomamos $n = 5$.

Congruencias verdaderas

- $13 \equiv 3 \pmod{5}$ porque $13 - 3 = 10 = 2 \cdot 5 \checkmark$
- $-2 \equiv 3 \pmod{5}$ porque $-2 - 3 = -5 = (-1) \cdot 5 \checkmark$
- $20 \equiv 0 \pmod{5}$ porque $20 - 0 = 20 = 4 \cdot 5 \checkmark$

Congruencia falsa

$7 \not\equiv 4 \pmod{5}$ porque $7 - 4 = 3$, que **no** es múltiplo de 5.

Regla práctica: $a \equiv b \pmod{n}$ si y solo si $a \bmod n = b \bmod n$.

Por ejemplo: $13 \bmod 5 = 3$ y $3 \bmod 5 = 3$. $\checkmark$ $7 \bmod 5 = 2 \neq 4 \bmod 5 = 4$. $\times$

Clases de equivalencia módulo n

La congruencia módulo n es una **relación de equivalencia** en $\mathbb{Z}$: agrupa a los enteros según su residuo al dividir por n .

Clase de equivalencia de r

$$[r] = \{ a \in \mathbb{Z} \mid a \equiv r \pmod{n} \} = \{ \dots, r - n, r, r + n, r + 2n, \dots \}$$

Las 5 clases módulo 5

Clase	Elementos
$[0]$	$\dots, -10, -5, 0, 5, 10, 15, \dots$
$[1]$	$\dots, -9, -4, 1, 6, 11, 16, \dots$
$[2]$	$\dots, -8, -3, 2, 7, 12, 17, \dots$
$[3]$	$\dots, -7, -2, 3, 8, 13, 18, \dots$
$[4]$	$\dots, -6, -1, 4, 9, 14, 19, \dots$

Hay exactamente n clases. El **representante canónico** de cada clase es su único elemento en $\{0, 1, \dots, n - 1\}$, devuelto por a mód n .

Suma y multiplicación de clases de equivalencia

Las clases de equivalencia módulo n se pueden **sumar y multiplicar** operando con sus representantes:

Definición

$$[a] + [b] = [a + b] \qquad [a] \cdot [b] = [a \cdot b]$$

¿**Están bien definidas**? El resultado no debe depender del representante elegido. Si $[a] = [a']$ y $[b] = [b']$, existen $k, l \in \mathbb{Z}$ tales que $a = a' + kn$ y $b = b' + ln$. Luego:

- **Suma:** $a + b = (a' + b') + (k + l)n \Rightarrow a + b \equiv a' + b' \pmod{n}$ ✓
- **Producto:** $ab = a'b' + n(a'l + b'k + kln) \Rightarrow ab \equiv a'b' \pmod{n}$ ✓

Ejemplo en $\mathbb{Z}_5$

$$[3] + [4] = [7] = [2] \quad \text{y} \quad [3] \cdot [4] = [12] = [2]$$

Usando 13 como representante de $[3]$: $[13] + [4] = [17] = [2]$ ✓ $[13] \cdot [4] = [52] = [2]$ ✓

$\mathbb{Z}_p$ como campo finito

Definición

$\mathbb{Z}_p = \{0, 1, \dots, p-1\}$ con suma y multiplicación **módulo** p : $a \oplus b = (a + b) \text{ mód } p$,
 $a \otimes b = (ab) \text{ mód } p$. Es un **campo** cuando p es primo.

Propiedades clave:

Operación	Propiedad
suma mód p	neutro 0, inverso $(-a) \text{ mód } p$
mult. mód p	neutro 1, inverso a^{-1} si $a \neq 0$

Ejemplo: $\mathbb{Z}_5$

a		0	1	2	3	4
a^{-1}		—	1	3	2	4
$2 \cdot 3 = 6 \equiv 1, \quad 4 \cdot 4 = 16 \equiv 1$						

¿Por qué p primo? Si n es compuesto, $\mathbb{Z}_n$ tiene *divisores de cero*: $2 \cdot 3 \equiv 0 \text{ (mód } 6)$, y algunos elementos no tienen inverso multiplicativo.

Teorema de Bézout

Teorema de Bézout

Para cualesquiera enteros a y b , existen enteros x e y tales que:

$$ax + by = \gcd(a, b)$$

En particular, si $\gcd(a, b) = 1$ (son **coprimos**), existen x, y tales que $ax + by = 1$.

Los enteros x e y se llaman **coeficientes de Bézout** y se calculan con el **algoritmo de Euclides extendido**.

Ejemplo: $a = 3$, $b = 7$

$\gcd(3, 7) = 1$. Euclides extendido: $7 = 2 \cdot 3 + 1$, luego $1 = 7 - 2 \cdot 3$.

Coefficientes: $x = -2$, $y = 1$. Verificar: $3 \cdot (-2) + 7 \cdot 1 = 1$. ✓

Inverso multiplicativo en $\mathbb{Z}_p$

Teorema

Para todo $a \in \{1, \dots, p-1\}$ existe un único $a^{-1} \in \mathbb{Z}_p$ tal que $a \cdot a^{-1} \equiv 1 \pmod{p}$.

Demostración (vía Bézout):

- ① Como p es primo y $1 \leq a \leq p-1$, se tiene $\gcd(a, p) = 1$.
- ② Por Bézout, existen enteros x, y tales que $ax + py = 1$.
- ③ Reduciendo módulo p : $py \equiv 0$, luego $ax \equiv 1 \pmod{p}$. $a^{-1} = x \pmod{p}$. □

Consecuencia clave

En $\mathbb{Z}_p$: $ab \equiv 0 \pmod{p}$ con $a \neq 0 \Rightarrow b \equiv 0 \pmod{p}$.

Prueba: multiplicar ambos lados por a^{-1} : $b \equiv a^{-1} \cdot 0 \equiv 0$. □

$h_{a,b}$ es una biyección (permutación) en $\mathbb{Z}_p$

Familia de funciones

$$h_{a,b} : \mathbb{Z}_p \rightarrow \mathbb{Z}_p, \quad h_{a,b}(x) = (ax + b) \text{ mód } p$$

con $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$ elegidos aleatoriamente.

Inyectividad: Si $h_{a,b}(x) = h_{a,b}(y)$, entonces $ax + b \equiv ay + b \pmod{p}$, luego $a(x - y) \equiv 0 \pmod{p}$. Como $a \neq 0$, la consecuencia clave da $x = y$. □

Biyectividad: Inyectiva en un conjunto finito $\Rightarrow$ también sobreyectiva $\Rightarrow$ **biyección**.

Inversa explícita: $h_{a,b}^{-1}(y) = a^{-1}(y - b) \text{ mód } p$ (existe porque a^{-1} existe en $\mathbb{Z}_p$).

Eficiencia

Almacenar $h_{a,b}$: solo 2 enteros (a, b) . Para $m = 100$ funciones: **200 enteros** en vez de $O(mN)$.

La familia $\mathcal{H}$ es 2-universal

Teorema

Para cualquier $x \neq y \in \mathbb{Z}_p$ y cualquier $u \neq v \in \mathbb{Z}_p$:

$$\Pr_{(a,b)}[h_{a,b}(x) = u \wedge h_{a,b}(y) = v] = \frac{1}{p(p-1)}$$

Demostración: Buscamos (a, b) , $a \in \mathbb{Z}_p^*$, que satisfagan $ax + b \equiv u$ y $ay + b \equiv v$ en $\mathbb{Z}_p$.

Paso 1 — a : Restando: $a(x - y) \equiv u - v$ (mód p). Como $x \neq y$, $(x - y)^{-1}$ existe, luego $a \equiv (u - v)(x - y)^{-1}$ (mód p), valor único. Como $u \neq v$, resulta $a \neq 0 \Rightarrow a \in \mathbb{Z}_p^*$. ✓

Paso 2 — b : Fijado a , la primera ecuación da $b \equiv u - ax$ (mód p), valor único. ✓

Exactamente **1 par** (a, b) válido de un total de $p(p-1)$, luego $\Pr = \frac{1}{p(p-1)}$. □

Consecuencia (ver slides siguientes)

El teorema implica **uniformidad** e **independencia por pares** de las prioridades asignadas.

Consecuencia 1: Uniformidad

Enunciado

Para cualquier elemento $x \in \mathbb{Z}_p$ y cualquier valor $u \in \mathbb{Z}_p$:

$$\Pr_{(a,b)}[h_{a,b}(x) = u] = \frac{1}{p}$$

Demostración: aplicamos la **ley de probabilidad total** condicionando en el valor de $h(y)$:

$$\Pr[h(x)=u] = \sum_{v \in \mathbb{Z}_p} \Pr[h(x)=u \wedge h(y)=v]$$

Separamos el término $v = u$ de los demás. Como h es **biyectiva** y $x \neq y$, es imposible que $h(x) = h(y)$, luego $\Pr[h(x)=u \wedge h(y)=u] = 0$. Para los $p - 1$ valores $v \neq u$, aplicamos el teorema:

$$\Pr[h(x)=u] = \underbrace{\Pr[h(x)=u \wedge h(y)=u]}_{=0} + \sum_{\substack{v \in \mathbb{Z}_p \\ v \neq u}} \underbrace{\Pr[h(x)=u \wedge h(y)=v]}_{=1/p(p-1)} = (p-1) \cdot \frac{1}{p(p-1)} = \frac{1}{p}$$

¿Qué significa en palabras?

Al elegir (a, b) al azar, el valor $h_{a,b}(x)$ es **equiprobable** entre los p posibles valores de $\mathbb{Z}_p$.

Dicho de otra forma: la función h le asigna a cada elemento una **prioridad aleatoria uniforme**, sin importar qué elemento sea ni cómo estén distribuidos los demás elementos.

Consecuencia 2: Independencia por pares

Enunciado

Para $x \neq y$ fijos y $v \neq u$, la **probabilidad condicional** de $h(y)$ dado $h(x)$ es:

$$\Pr[h(y) = v \mid h(x) = u] = \frac{1}{p-1}$$

Demostración: aplicamos la definición de **probabilidad condicional**:

$$\Pr[h(y)=v \mid h(x)=u] = \frac{\Pr[h(x)=u \wedge h(y)=v]}{\Pr[h(x)=u]} = \frac{1/p(p-1)}{1/p} = \frac{1}{p-1}$$

donde usamos el **teorema 2-universal** en el numerador y la **consecuencia 1** en el denominador.

¿Qué significa en palabras?

- **Sin** saber $h(x)$: $h(y)$ es uniforme en p valores $\rightarrow$ probabilidad $\frac{1}{p}$ cada uno.
- **Sabiendo** $h(x) = u$: $h(y)$ es uniforme en los $p-1$ valores restantes $\rightarrow$ probabilidad $\frac{1}{p-1}$ cada uno.

La distribución de $h(y)$ cambia *solo* porque el valor u queda descartado por la biyección — **no hay ninguna otra influencia de $h(x)$ sobre $h(y)$** .

¿Por qué importan estas dos consecuencias?

$h(A) = h(B)$ ocurre cuando el elemento con **menor valor** h en $A \cup B$ pertenece a $A \cap B$.

Por **uniformidad**: cada $x \in A \cup B$ tiene la misma probabilidad $\frac{1}{p}$ de recibir cualquier prioridad.

Por **independencia por pares**: la prioridad de un elemento no influye en la de los demás (más allá de que sean distintas). Ningún elemento está sistemáticamente favorecido.

Por tanto, cada $x \in A \cup B$ es igualmente probable de tener el mínimo, y el mínimo cae en $A \cap B$ con probabilidad:

$$\Pr[h(A) = h(B)] = \Pr\left[\min_{x \in A \cup B} h(x) \in A \cap B\right] = \frac{|A \cap B|}{|A \cup B|} = \text{SIM}(A, B)$$

Intuición: de permutaciones a mínimos

Dada una matriz característica, con una permutación π de sus filas, el minhash de S es el **primer elemento** de S en el orden π . Equivalentemente: el elemento $x \in S$ cuya posición $\pi(x)$ es la más pequeña:

$$h_{\pi}(S) = \arg \min_{x \in S} \pi(x)$$

Nuestra función hash h asigna a cada fila un “número de prioridad”. Podemos pensar que h **induce un orden** sobre las filas: la fila con menor valor $h(x)$ va primero.

Observación clave

El minhash con h es simplemente el **mínimo** valor hash entre los elementos de S :

$$\text{sig}_h(S) = \min_{x \in S} h(x)$$

No necesitamos ordenar ni permutar — solo calcular $h(x)$ para cada $x \in S$ y quedarnos con el mínimo.

Este mínimo se puede calcular **incrementalmente**: procesamos los elementos uno por uno, manteniendo el mínimo visto hasta ahora.

Algoritmo de construcción de firmas

La observación anterior nos da un algoritmo eficiente para construir **todas las firmas a la vez**, recorriendo la matriz característica fila por fila:

Algoritmo

- 1 Inicializar $\text{sig}[i][c] = \infty$ para toda función hash i y columna c
- 2 Para cada fila r :
 - Calcular $h_1(r), h_2(r), \dots, h_m(r)$
 - Para cada columna c con un 1 en fila r : (es decir, $r \in S_c$)

$$\text{sig}[i][c] \leftarrow \min(\text{sig}[i][c], h_i(r))$$

Al terminar, $\text{sig}[i][c] = \min_{r \in S_c} h_i(r)$ — exactamente el minhash de S_c bajo h_i .

Eficiencia:

- Solo **una pasada** sobre los datos
- Para cada fila, solo tocamos columnas con un 1 (la matriz es **dispersa**)
- Costo total: $O(m \cdot N_{\text{nnz}})$ donde N_{nnz} = número de entradas no-cero

Ejemplo: datos y funciones hash

Retomemos la matriz característica con $S_1 = \{a, d\}$, $S_2 = \{c\}$, $S_3 = \{b, d, e\}$, $S_4 = \{a, c, d\}$. Numeramos las filas: $a=0, b=1, c=2, d=3, e=4$. Elegimos $p = 5$ (primo) y dos funciones hash:

$$h_1(x) = (x + 1) \bmod 5, \quad h_2(x) = (3x + 1) \bmod 5$$

	Fila	S_1	S_2	S_3	S_4
a	0	1	0	0	1
b	1	0	0	1	0
c	2	0	1	0	1
d	3	1	0	1	1
e	4	0	0	1	0

Fila x	$h_1(x)$	$h_2(x)$
0	1	1
1	2	4
2	3	2
3	4	0
4	0	3

Inicializamos: $\text{sig}[i][c] = \infty$ para todo $i \in \{1, 2\}$ y $c \in \{S_1, S_2, S_3, S_4\}$.

Ejemplo paso a paso: inicialización

	Fila	S_1	S_2	S_3	S_4
a	0	1	0	0	1
b	1	0	0	1	0
c	2	0	1	0	1
d	3	1	0	1	1
e	4	0	0	1	0

Matriz de firmas (inicializada en ∞):

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

Regla: para cada fila r , calculamos $h_1(r)$ y $h_2(r)$. Para cada columna c que tenga un **1** en la fila r , actualizamos:

$$\text{sig}[i][c] \leftarrow \min(\text{sig}[i][c], h_i(r))$$

Las columnas con **0** en la fila r no se tocan.

Ejemplo paso a paso: fila a ($x = 0$)

Calculamos: $h_1(0) = (0 + 1) \bmod 5 = \mathbf{1}$, $h_2(0) = (3 \cdot 0 + 1) \bmod 5 = \mathbf{1}$

¿Qué columnas tienen un 1 en la fila a ? S_1 y S_4 .

Actualizamos solo esas columnas:

- $\text{sig}[\cdot][S_1] \leftarrow \min(\infty, 1) = 1$ para h_1 y h_2
- $\text{sig}[\cdot][S_4] \leftarrow \min(\infty, 1) = 1$ para h_1 y h_2

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

S_2 y S_3 no se tocan porque tienen 0 en la fila a .

Ejemplo paso a paso: fila b ($x = 1$)

Calculamos: $h_1(1) = (1 + 1) \bmod 5 = 2$, $h_2(1) = (3 \cdot 1 + 1) \bmod 5 = 4$

¿Qué columnas tienen un 1 en la fila b ? Solo S_3 .

Actualizamos:

- $\text{sig}[h_1][S_3] \leftarrow \min(\infty, 2) = 2$
- $\text{sig}[h_2][S_3] \leftarrow \min(\infty, 4) = 4$

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

Ejemplo paso a paso: fila c ($x = 2$)

Calculamos: $h_1(2) = (2 + 1) \bmod 5 = 3$, $h_2(2) = (3 \cdot 2 + 1) \bmod 5 = 2$

¿Qué columnas tienen un 1 en la fila c ? S_2 y S_4 .

Actualizamos:

- S_2 : $\text{sig}[h_1][S_2] \leftarrow \min(\infty, 3) = 3$; $\text{sig}[h_2][S_2] \leftarrow \min(\infty, 2) = 2$
- S_4 : $\text{sig}[h_1][S_4] \leftarrow \min(1, 3) = 1$; $\text{sig}[h_2][S_4] \leftarrow \min(1, 2) = 1$
→ S_4 **no cambia**: los valores actuales ya son menores

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

Ejemplo paso a paso: fila d ($x = 3$)

Calculamos: $h_1(3) = (3 + 1) \bmod 5 = \mathbf{4}$, $h_2(3) = (3 \cdot 3 + 1) \bmod 5 = \mathbf{0}$

¿Qué columnas tienen un 1 en la fila d ? S_1 , S_3 y S_4 .

Actualizamos (el mín solo baja, nunca sube):

- S_1 : h_1 : $\min(1, 4) = 1$; h_2 : $\min(1, \mathbf{0}) = \mathbf{0}$ (h_2 baja de 1 a 0)
- S_3 : h_1 : $\min(2, 4) = 2$; h_2 : $\min(4, \mathbf{0}) = \mathbf{0}$ (h_2 baja de 4 a 0)
- S_4 : h_1 : $\min(1, 4) = 1$; h_2 : $\min(1, \mathbf{0}) = \mathbf{0}$ (h_2 baja de 1 a 0)

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	0	2	0	0

Ejemplo paso a paso: fila e ($x = 4$)

Calculamos: $h_1(4) = (4 + 1) \bmod 5 = \mathbf{0}$, $h_2(4) = (3 \cdot 4 + 1) \bmod 5 = \mathbf{3}$

¿Qué columnas tienen un 1 en la fila e? Solo S_3 .

Actualizamos:

- S_3 : h_1 : $\min(2, \mathbf{0}) = \mathbf{0}$; h_2 : $\min(0, 3) = 0$ (h_1 baja de 2 a 0)

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Resultado final — matriz de firmas

$$\text{sig}(S_1) = [1, 0], \quad \text{sig}(S_2) = [3, 2], \quad \text{sig}(S_3) = [0, 0], \quad \text{sig}(S_4) = [1, 0]$$

Ejemplo: verificación de similitudes

Comparamos las firmas para estimar la similitud de Jaccard:

Par	Coincidencias	$\hat{p}$ (estimada)	SIM (real)
(S_1, S_4)	$h_1: 1=1 \checkmark, h_2: 0=0 \checkmark$	$2/2 = 1,0$	$2/3 \approx 0,67$
(S_1, S_3)	$h_1: 1 \neq 0 \text{ X}, h_2: 0=0 \checkmark$	$1/2 = 0,5$	$1/4 = 0,25$
(S_1, S_2)	$h_1: 1 \neq 3 \text{ X}, h_2: 0 \neq 2 \text{ X}$	$0/2 = 0,0$	$0/3 = 0,0$

Con solo $m = 2$ funciones hash, las estimaciones son **imprecisas**.

Pero la tendencia es correcta: $\hat{p}(S_1, S_4) > \hat{p}(S_1, S_3) > \hat{p}(S_1, S_2)$.

Con $m = 100\text{--}200$, las estimaciones se acercan mucho a los valores reales (ley de los grandes números).

El problema restante

Aun con firmas compactas, comparar **todos los pares** sigue siendo $O(n^2)$.

Ejemplo

- 100 000 documentos, firmas de longitud 100 (enteros de 4 bytes)
- Matriz de firmas: 40 MB — cabe en memoria
- Pero: $\binom{10^5}{2} \approx 5 \times 10^9$ pares que comparar

Solución: Locality-Sensitive Hashing (LSH)

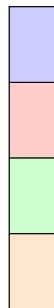
Encontrar **pares candidatos** probablemente similares, examinando solo una fracción de todos los pares.

Técnica de bandas — idea

Idea

Dividir la firma de longitud n en b **bandas** de r **filas** ($n = b \cdot r$).

Firma



Banda 1 ($\overset{\text{hash}}{\underset{\text{filas}}{r}}$) buckets banda 1

Banda 2 ($\overset{\text{hash}}{\underset{\text{filas}}{r}}$) buckets banda 2

Banda 3 ($\overset{\text{hash}}{\underset{\text{filas}}{r}}$) buckets banda 3

Banda b (r filas)

Para cada banda, aplicamos una **función hash** al sub-vector de r componentes. Documentos con el mismo hash en una banda van al mismo **bucket**.

Técnica de bandas — mecanismo

Para cada banda j ($j = 1, \dots, b$):

- 1 Extraer las r componentes de la banda j de cada firma
- 2 Usar el sub-vector como **clave** en una tabla hash $\rightarrow$ asignar un **bucket**
- 3 Documentos con sub-vector **idéntico** $\rightarrow$ mismo bucket $\rightarrow$ candidatos

¿Qué es un bucket?

Un grupo en una tabla hash. El sub-vector de r componentes es la clave: sub-vector idéntico $\rightarrow$ mismo bucket. Basta que **una sola** componente difiera $\rightarrow$ buckets diferentes.

Regla de candidatura

Dos docs son **pares candidatos** si caen en el mismo bucket en ≥ 1 de las b bandas.

Intuición:

- Docs **muy similares**: muchas componentes iguales $\rightarrow$ alta prob. de coincidir en alguna banda
- Docs **poco similares**: difícil que **todas** las r componentes coincidan

Técnica de bandas — ejemplo

6 documentos, firmas de longitud $n = 8$, divididas en $b = 4$ bandas de $r = 2$:

	D_1	D_2	D_3	D_4	D_5	D_6
h_1	1	1	0	3	2	4
h_2	3	3	1	0	5	2
h_3	2	0	4	2	3	1
h_4	1	3	0	1	4	3
h_5	0	2	3	5	0	1
h_6	4	1	2	3	4	5
h_7	3	5	1	2	0	1
h_8	2	0	4	0	3	4

Buckets con ≥ 2 docs (los demás caen solos):

Banda 1: $[1, 3] \rightarrow \{D_1, D_2\}$

Banda 2: $[2, 1] \rightarrow \{D_1, D_4\}$

Banda 3: $[0, 4] \rightarrow \{D_1, D_5\}$

Banda 4: $[1, 4] \rightarrow \{D_3, D_6\}$

Los otros 20 sub-vectores son únicos $\rightarrow$ buckets con un solo documento.

Pares candidatos: 4 de $\binom{6}{2} = 15$ pares

$(D_1, D_2), (D_1, D_4), (D_1, D_5), (D_3, D_6)$. **11 pares no examinados (73 %).**

¿Por qué las bandas ahorran trabajo?

Sin bandas (fuerza bruta):

- Comparar **todos** los $\binom{n}{2}$ pares de firmas
- Cada comparación: revisar las m componentes
- Costo total: $O(n^2 \cdot m)$

Con $n = 10^5$ y $m = 100$:

$5 \times 10^9 \times 100 = 5 \times 10^{11}$ operaciones

Con bandas (LSH):

- Para cada banda, hacer hash de cada firma $\rightarrow$ bucket: $O(n)$
- Solo comparar pares en el **mismo bucket**
- Costo total: $O(b \cdot n + \text{candidatos})$

Los candidatos son típicamente $\ll \binom{n}{2}$, porque docs poco similares casi nunca caen en el mismo bucket.

Idea clave

Las bandas actúan como un **filtro**: el hash agrupa documentos en buckets, y solo pares en el mismo bucket se comparan. No necesitamos examinar los $\binom{n}{2}$ pares.

Punto de partida: la firma como secuencia de Bernoullis

Tenemos firmas de longitud n . Para un par (C_1, C_2) con similitud $t = \text{SIM}(C_1, C_2)$:
Cada componente i de la firma coincide **independientemente** con probabilidad t :

$$\Pr[h_i(C_1) = h_i(C_2)] = t$$

Tenemos n experimentos de Bernoulli independientes. La pregunta es:

Pregunta clave

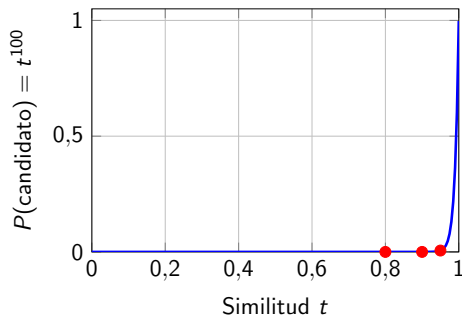
¿Cómo usamos estos n resultados para decidir si (C_1, C_2) es un **par candidato**?

Veremos que hay dos extremos, ambos malos, y que la técnica de bandas encuentra el **equilibrio justo**.

Intento 1: una sola banda ($b = 1$, $r = n$)

Criterio: exigir que **todas** las n componentes coincidan.

Como las componentes son independientes: $P(\text{candidato}) = t^n$



t	t^{100}
0.50	≈ 0
0.80	2×10^{-10}
0.90	$2,7 \times 10^{-5}$
0.95	0,006
0.99	0,366

Incluso con $t = 0,95$, la probabilidad de ser candidato es **0.6 %**.

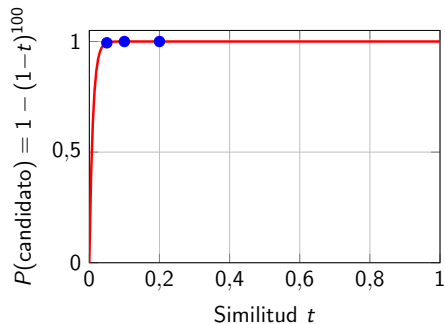
Problema: demasiado estricto

El AND de n eventos hace que P decaiga **exponencialmente**. Solo pares casi idénticos ($t > 0,99$) pasan el filtro.

Intento 2: cada fila es una banda ($b = n, r = 1$)

Criterio: ser candidatos si **al menos una** componente coincide.

Complemento: $P(\text{candidato}) = 1 - (1 - t)^n$



t	$1 - (1-t)^{100}$
0.01	0,634
0.05	0,994
0.10	≈ 1
0.20	≈ 1
0.50	≈ 1

Incluso con $t = 0,01$, la probabilidad de ser candidato es **63 %**.

Problema: demasiado permisivo

El OR de n eventos hace que P suba a 1 para cualquier $t > 0$. Casi **todos** los pares son candidatos — no filtra nada.

El dilema AND vs. OR

Estrategia	Fórmula	Problema
AND puro ($b=1, r=n$)	t^n	demasiado estricto
OR puro ($b=n, r=1$)	$1 - (1 - t)^n$	demasiado permisivo

Observación

- **AND** reduce P (exige que *todos* ocurran)
- **OR** aumenta P (basta con que *uno* ocurra)

Idea: combinar AND y OR

- 1 **Dentro de banda:** AND de $r \rightarrow t^r$
- 2 **Entre bandas:** OR de $b \rightarrow 1 - (1 - t^r)^b$

Con r pequeño el AND no es tan estricto; con b grande el OR da suficientes oportunidades. El resultado es una **curva S** que aproxima la función escalón ideal.

Derivación de la fórmula, paso a paso

Sea un par (C_1, C_2) con similitud t , firma de longitud $n = b \cdot r$.

- ① **Una componente coincide** (propiedad del minhash):

$$\Pr[h_i(C_1) = h_i(C_2)] = t$$

- ② **Una banda concreta coincide**: sus r componentes deben coincidir **todas** (AND). Como son independientes, las probabilidades se multiplican:

$$\Pr[\text{banda } j \text{ coincide}] = \underbrace{t \cdot t \cdots t}_{r \text{ veces}} = t^r$$

- ③ **Esa banda no coincide** (complemento):

$$\Pr[\text{banda } j \text{ no coincide}] = 1 - t^r$$

- ④ **Ninguna** de las b bandas coincide: las bandas usan funciones hash distintas $\rightarrow$ independientes, así que de nuevo se multiplican:

$$\Pr[\text{ninguna banda coincide}] = (1 - t^r)^b$$

Derivación de la fórmula: el último paso

Paso 5. Ser par candidato = coincidir en **al menos una** banda (OR). Es el complemento del paso 4:

Fórmula de la curva S

$$P(\text{candidato}) = 1 - (1 - t^r)^b$$

Verificación: $b=20$, $r=5$, $t=0,8$

$$t^r = 0,8^5 = 0,328$$

$$1 - t^r = 0,672$$

$$(1 - t^r)^b = 0,672^{20} = 0,00035$$

$$P = 1 - 0,00035 = \mathbf{0,9996}$$

Control de sanidad: los dos extremos

- $b=1$, $r=n$: $1 - (1 - t^n)^1 = t^n$ (AND puro)
- $b=n$, $r=1$: $1 - (1 - t)^n$ (OR puro)

Supuestos: independencia entre componentes y entre bandas; se ignoran las colisiones de hash (dos sub-vectores *distintos* casi nunca caen en el mismo bucket).

De AND puro a AND+OR: paso a paso

Supongamos $n = 100$ y $t = 0,8$:

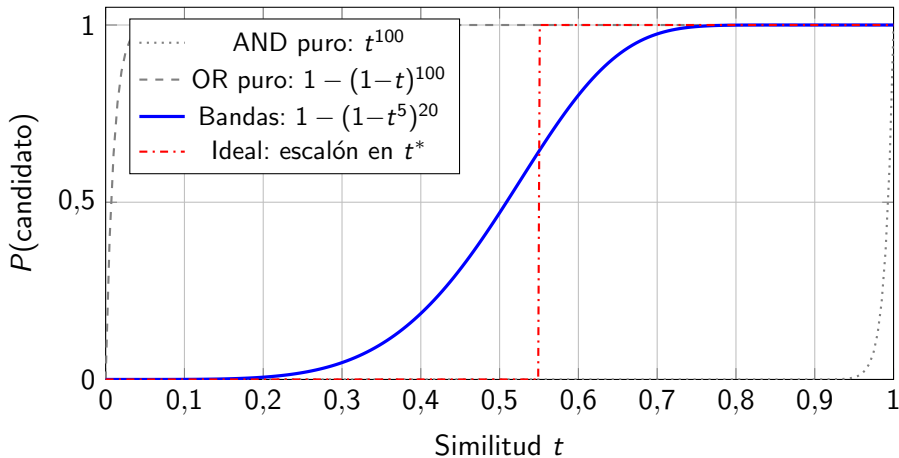
Configuración	Fórmula	P para $t=0,8$	P para $t=0,2$
$b=1, r=100$ (AND puro)	t^{100}	≈ 0	≈ 0
$b=2, r=50$	$1 - (1 - t^{50})^2$	0,028	≈ 0
$b=5, r=20$	$1 - (1 - t^{20})^5$	0,058	≈ 0
$b=10, r=10$	$1 - (1 - t^{10})^{10}$	0,722	≈ 0
$b=20, r=5$	$1 - (1 - t^5)^{20}$	0,9996	0,006
$b=50, r=2$	$1 - (1 - t^2)^{50}$	≈ 1	0,982
$b=100, r=1$ (OR puro)	$1 - (1 - t)^{100}$	≈ 1	≈ 1

Punto clave

$b=20, r=5$ captura el 99.96 % de pares con $t=0,8$ y solo el 0.6 % de pares con $t=0,2$.

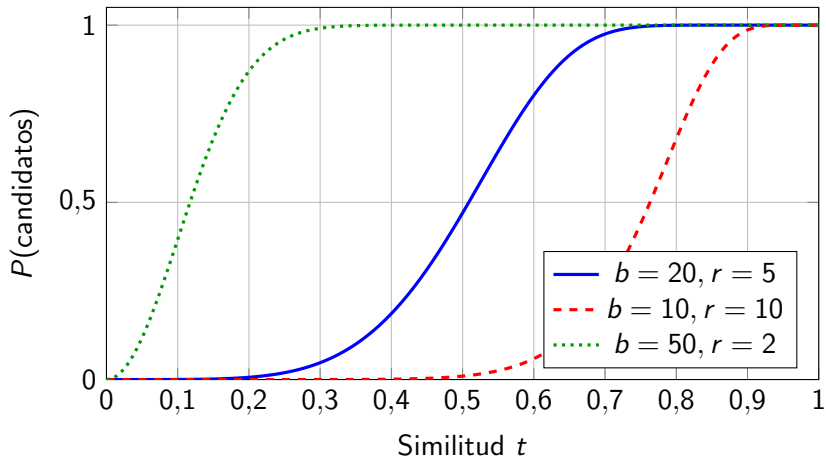
Es la **combinación justa** de AND (filtro por banda) y OR (múltiples oportunidades).

Visualización: AND puro vs. OR puro vs. bandas



La técnica de bandas ($b=20$, $r=5$) **aproxima la función escalón ideal**: probabilidad ≈ 0 para pares poco similares, ≈ 1 para pares similares.

La curva S para distintos b y r



Todas con $n = b \cdot r = 100$ funciones hash, pero distinto balance.

El umbral aproximado: ¿de dónde sale?

Idea: contar cuántas bandas coinciden, en promedio.

- 1 Cada banda coincide con probabilidad t^r , y hay b bandas. El **número esperado de bandas que coinciden** es

$$\mathbb{E}[\text{\#bandas que coinciden}] = b \cdot t^r$$

- 2 El umbral es donde ese valor esperado **cruza 1**: por debajo lo típico es que *ninguna* banda coincida (no candidato); por encima se espera *al menos una* (candidato):

$$b \cdot (t^*)^r = 1$$

- 3 Despejando: $(t^*)^r = \frac{1}{b} \implies t^* \approx \left(\frac{1}{b}\right)^{1/r}$

¿Por qué $\approx$?

En la curva S: $P(t^*) = 1 - \left(1 - \frac{1}{b}\right)^b \approx 1 - e^{-1} \approx 0,63$ para b grande. O sea, t^* cae en plena subida (al 63%), pero no es el punto de inflexión exacto.

El umbral aproximado: lectura

$$t^* \approx \left(\frac{1}{b}\right)^{1/r}$$

- Pares con similitud $t \gg t^* \Rightarrow$ candidatos con **alta probabilidad**
- Pares con similitud $t \ll t^* \Rightarrow$ candidatos con **baja probabilidad**

Ejemplo: $b = 20$, $r = 5$ ($n = 100$)

$$t^* \approx (1/20)^{1/5} \approx 0,55$$

En la práctica

Se fija primero el umbral s que interesa al problema y luego se eligen b y r (con $n = b \cdot r$) para situar t^* **algo por debajo** de s — ver más adelante por qué no exactamente en s .

Falsos positivos y falsos negativos

- **Falso negativo:** par con similitud $\geq s$ que *no* se identifica como candidato (ninguna banda coincide a pesar de ser similares)
- **Falso positivo:** par con similitud $< s$ que *sí* se identifica como candidato (coinciden por azar en alguna banda)

Control del balance ajustando b y r :

- r grande (más filas por banda) $\Rightarrow$ curva S a la **derecha**
 - ⊕ Menos falsos positivos
 - ⊖ Más falsos negativos
- b grande (más bandas) $\Rightarrow$ curva S a la **izquierda**
 - ⊕ Menos falsos negativos
 - ⊖ Más falsos positivos

Del umbral s a los parámetros: la receta

Hasta ahora fuimos de b, r hacia la curva. En la práctica se va **al revés**.

- 1 **El umbral s lo fija el problema**, no el algoritmo (plagio: $s \approx 0,9$; noticias parecidas: $s \approx 0,6$).
- 2 **Elegir el presupuesto n** (longitud de firma). Es el mando de *nitidez*: n grande $\Rightarrow$ la S se parece más al escalón $\Rightarrow$ menos errores de ambos tipos, a costa de memoria y tiempo.
- 3 **Factorizar $n = b \cdot r$** eligiendo la combinación que sitúe $t^* \approx (1/b)^{1/r}$ donde interesa.
- 4 **Ajustar según el error que más duela** (ver la diapositiva siguiente).
- 5 **Ejecutar**: firmas $\rightarrow$ bandas $\rightarrow$ buckets $\rightarrow$ pares candidatos.
- 6 **Verificar**: calcular SIM real de cada candidato y quedarse con los $\geq s$.

El umbral se usa dos veces

Paso 3 (aproximado): s guía la elección de b y r — t^* solo dice dónde se dobla la curva S .

Paso 6 (exacto): s es la comparación dura que decide la respuesta.

LSH nunca aplica el umbral: lo **aproxima**. El umbral de verdad se aplica al verificar.

Ajustar b y r : ejemplo con $s = 0,8$

Opciones con $n \approx 100$:

b	r	t^*	$P(0,6)$	$P(0,7)$	$P(0,8)$	$P(0,9)$
20	5	0.55	0.80	0.98	1.00	1.00
14	7	0.69	0.33	0.70	0.96	1.00
12	8	0.73	0.18	0.51	0.89	1.00
10	10	0.79	0.06	0.25	0.68	0.99
8	12	0.84	0.02	0.11	0.43	0.93

Poner $t^* = s$ *no* es lo que queremos

$b=10, r=10$ da $t^* = 0,79 \approx s$, pero $P(0,8) = 0,68$: se pierde el **32 %** de los pares que están justo en el umbral. Es el $1 - e^{-1} \approx 0,63$ que ya vimos.

Solución: bajar t^* por debajo de s (subiendo n)

$b=20, r=7$ ($n=140, t^*=0,65$): $P(0,8) = 0,99, P(0,6) = 0,43$

$b=25, r=8$ ($n=200, t^*=0,67$): $P(0,8) = 0,99, P(0,6) = 0,35, P(0,5) = 0,09$

Regla práctica: apuntar t^* unos 0,1–0,15 por debajo de s .

Resumen del algoritmo LSH

- 1 Elegir M funciones minhash. Calcular la **firma** de cada documento.
- 2 Dividir firmas en b bandas de r filas ($M = b \cdot r$).
- 3 Para cada banda, aplicar hash a las porciones de firma $\rightarrow$ buckets.
- 4 Pares en el mismo bucket en ≥ 1 banda = **pares candidatos**.
- 5 **Verificar** candidatos: calcular su similitud real y conservar los que cumplen $\text{SIM} \geq s$.
- 6 (Opcional) Pasada final sobre datos originales para confirmar.

Clave

El costo es proporcional al número de **pares candidatos**, típicamente $\ll O(n^2)$.

Medidas de distancia

LSH no se limita a la similitud de Jaccard. Existen familias LSH para diversas medidas de distancia.

Axiomas de una métrica $d(x, y)$

- 1 $d(x, y) \geq 0$, con $d(x, y) = 0 \iff x = y$
- 2 $d(x, y) = d(y, x)$
- 3 $d(x, z) \leq d(x, y) + d(y, z)$ (desigualdad triangular)

Distancias comunes:

- **Euclidiana** (L_2): $\sqrt{\sum (x_i - y_i)^2}$
- **Manhattan** (L_1): $\sum |x_i - y_i|$
- **Coseno**: ángulo θ entre vectores
- **Hamming**: posiciones que difieren en vectores binarios
- **Edición**: inserciones/eliminaciones entre cadenas

Familias (d_1, d_2, p_1, p_2) -sensibles

Definición

Una familia $\mathcal{F}$ de funciones hash es (d_1, d_2, p_1, p_2) -sensible si para $f \in \mathcal{F}$:

- $d(x, y) \leq d_1 \Rightarrow \Pr[f(x) = f(y)] \geq p_1$
- $d(x, y) \geq d_2 \Rightarrow \Pr[f(x) = f(y)] \leq p_2$

con $d_1 < d_2$ y $p_1 > p_2$.

Amplificación con construcciones AND y OR:

- **AND** (r funciones): (d_1, d_2, p_1^r, p_2^r) — reduce falsos positivos
- **OR** (b funciones): $(d_1, d_2, 1 - (1 - p_1)^b, 1 - (1 - p_2)^b)$ — reduce falsos negativos

La **técnica de bandas** es exactamente AND seguido de OR.

Familias LSH para cada distancia

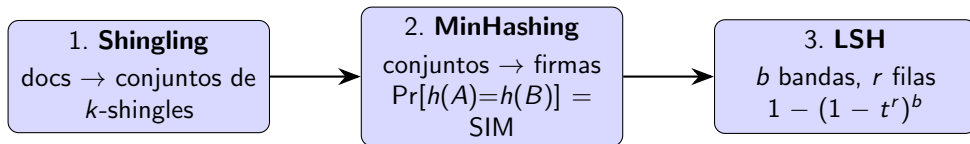
Distancia	Familia LSH	$\Pr[f(x) = f(y)]$
Jaccard	MinHash	$1 - d_J$
Hamming	Componente aleatoria $f_i(x) = x_i$	$1 - d_H/n$
Coseno	Hiperplano aleatorio	$1 - \theta/\pi$
Euclidiana	Proyección + segmentos	depende de d/a

Coseno — hiperplanos aleatorios:

- Vector aleatorio $\mathbf{v}$ define un hiperplano por el origen
- $f_{\mathbf{v}}(\mathbf{x}) = \text{sign}(\mathbf{v} \cdot \mathbf{x})$; prob. de separar $\mathbf{x}$ e $\mathbf{y}$ con ángulo θ : θ/π

Euclidiana — proyección:

- Proyectar sobre línea aleatoria, dividir en segmentos de longitud a
- Familia $(a/2, 2a, 1/2, 1/3)$ -sensible



- El framework LSH generaliza a cualquier distancia usando familias (d_1, d_2, p_1, p_2) -sensibles
- Las construcciones AND/OR amplifican la separación entre p_1 y p_2
- Resultado: búsqueda de similitud en tiempo **sublineal** respecto a n^2